
Inductive Detection of Illicit Bitcoin Transactions via Temporal Graph Features

Bryan Fang¹ Jaray Liu¹ Evan Ho¹ Andrew Lee¹

Abstract

Detecting illicit transactions on the Bitcoin blockchain is a longstanding challenge for anti-money laundering, but the field’s standard benchmark, the Elliptic dataset, is evaluated almost exclusively under a transductive protocol that lets models observe the entire graph at training time and uses node features computed over each transaction’s full lifetime. Both choices leak future information into present predictions, and the resulting numbers substantially overstate how a deployed system would actually perform. We make three methodological contributions. First, we formalize a strict-inductive task with walk-forward evaluation under which only features, edges, and labels with timestamp strictly less than the prediction step are admissible, and we show that several empirical phenomena, including a large interaction between graph features and retraining, are invisible under transductive evaluation but emerge clearly under ours. Second, we release **Elliptic++Causal**, a regeneration of the Elliptic++ actor-side data in which wallet features are recomputed as cumulative aggregates and each of the 2.87M wallet-wallet edges is annotated with the first timestep at which the two wallets co-transacted, together with three families of strictly causal cross-timestep features (wallet trajectories, money-flow pair statistics, and illicit-seeded personalized PageRank). Third, we benchmark tree-based models, vanilla GCN baselines, and heterogeneous GNNs on the proposed task. Our engineered features lift heterogeneous GNN F1 by roughly +0.16 and produce a headline walk-forward LightGBM at $F1 = 0.856$ versus a strict no-graph floor of 0.802, a +0.054 lift that decomposes into +0.013 from graph features alone, +0.003 from inductive walk-forward retraining alone, and +0.037 from their interaction.

1. Introduction

The pseudonymous nature of Bitcoin has made it an attractive vehicle for financial crime. Although all Bitcoin trans-

actions are recorded on a public ledger, linking addresses to real-world identities remains difficult, enabling money laundering, ransomware payments, and darknet marketplace activity at scale. Financial institutions collectively spend more than \$25 billion annually on anti-money laundering compliance, yet a significant share of illicit flows continues to evade detection (Weber et al., 2019b).

Graph-based representations are a natural fit for blockchain transaction data: each transaction can be modeled as a node, with directed edges encoding flows of funds between transactions. The Elliptic dataset (Weber et al., 2019a) provides such a graph comprising roughly 203K transaction nodes, 234K edges, and 166 hand-crafted features, along with partial labels (approximately 2% illicit, 21% licit, and 77% unlabeled) spread across 49 timesteps. Its successor, Elliptic++ (Elmougy & Liu, 2023), augments the graph with additional address- and entity-level information and richer edge types.

Despite the graph structure, the current state-of-the-art on the Elliptic benchmark is dominated by Random Forest models that treat each transaction independently, leveraging only the 166 precomputed node features (Weber et al., 2019a). Standard graph neural network (GNN) approaches, including GCN (Kipf & Welling, 2017) and GAT (Veličković et al., 2018), have consistently underperformed relative to feature-engineered tree-based methods on this dataset. Moreover, nearly all published results adopt a *transductive* evaluation protocol in which the model observes the entire graph (including test-time nodes) during training, and features may encode lifetime statistics that would be unavailable at the point of an actual detection decision.

Contributions. We make four contributions, of which the first two are the central methodological advances.

1. **Task: strict-inductive detection with walk-forward evaluation.** We formalize a deployment-faithful evaluation contract for illicit-transaction detection on Elliptic and Elliptic++. At every prediction step T , only features, edges, and historical labels with source timestamp strictly less than T are admissible. We define two evaluation protocols within this contract, a static-causal split for direct comparison with prior work and a walk-forward protocol that retrains the model as labels

arrive, and argue that both should be standard for any future Elliptic results that claim deployment relevance.

2. **Dataset: Elliptic++Causal.** We release a regenerated version of the Elliptic++ actor-side data that is admissible under the strict-causal contract. The wallet feature file is rebuilt so each (wallet, timestep) row contains cumulative aggregates over $\tau \leq T$ rather than lifetime aggregates; each of the 2.87M wallet-wallet edges in the original `AddrAddr_edgelist.txt` is annotated with a `first_t` field equal to the earliest timestep at which the two wallets co-transacted in the appropriate input/output orientation, allowing the edge to be admitted only at $t \geq \text{first}_t$. To this regenerated graph we attach three families of strictly causal cross-timestep features per transaction: 103 per-wallet trajectory aggregates, 17 directed money-flow pair statistics, and 5 personalized-PageRank features seeded from the historical illicit subgraph.
3. **Initial baselines.** We provide the first reported numbers on the proposed task across four model families: tree-based classifiers (Random Forest, LightGBM) on the engineered features, vanilla and skip-connected GCN on the original 166 features, and three heterogeneous GNN architectures (GCN, GAT, GraphSAGE) operating on the Elliptic++Causal graph. Across model families, the engineered causal features deliver large lifts: roughly +0.16 F1 for heterogeneous GNNs and +0.013 F1 for the headline tree model (rising to +0.054 with walk-forward retraining), with walk-forward retraining contributing additional gains.
4. **Empirical analysis of task framing.** We demonstrate that the contributions of graph features and walk-forward retraining interact synergistically: graph features alone add +0.013 F1, walk-forward retraining alone adds +0.003, but their combination adds +0.054, yielding a synergy term of +0.038 that only becomes visible once the task is correctly defined. The model’s ability to absorb distribution drift through retraining is what unlocks the deployment value of the engineered graph features.

Code and data availability. All code, the regenerated Elliptic++Causal dataset, and the trained model checkpoints needed to reproduce every number in this paper are available at <https://github.com/jarayl/stat175-final>.

2. Background and Related Work

2.1. Bitcoin Transaction Graphs

A Bitcoin transaction consumes one or more *unspent transaction outputs* (UTXOs) and produces new outputs that

can be spent by future transactions. The resulting directed acyclic graph (DAG) encodes the flow of value over time. Each node in the Elliptic dataset corresponds to a single transaction and is annotated with 166 features: 94 *local* features (e.g., transaction value, fee, number of inputs/outputs) and 72 *aggregated neighborhood* features (e.g., statistics over one-hop neighbors). Edges represent flows of Bitcoin between consecutive transactions. Transactions are further grouped into 49 timesteps, each spanning approximately two weeks.

2.2. The Elliptic and Elliptic++ Datasets

The Elliptic dataset (Weber et al., 2019a) was the first large-scale labeled Bitcoin transaction graph released for research; we describe it in detail in Section 3. Elliptic++ (Elmougy & Liu, 2023) extends it by introducing a heterogeneous graph with four distinct edge types, address-level nodes, and entity-level information that enables richer relational reasoning.

2.3. Prior Approaches

Weber et al. (2019a) established the benchmark with a Random Forest classifier operating on the 166-dimensional feature vectors, achieving strong performance without any explicit graph learning. Subsequent work has largely focused on improving feature representations or applying GNN architectures. However, GCN and GAT models have consistently lagged behind the Random Forest baseline on this benchmark, a phenomenon attributed to the high informativeness of the pre-aggregated neighborhood features and the relatively sparse, noisy graph structure.

A known failure mode of all published models is a sharp performance drop around timestep 43, corresponding to a regime change in the data distribution, likely caused by a massive dark-market takedown that occurred in this time period.

2.4. Transductive vs. Inductive Evaluation

Most prior work evaluates models in a *transductive* setting, where the full graph (including test nodes) is available at training time, and features may encode statistics computed over the node’s entire lifetime. This setup is unrealistic for deployment: in practice, a model must classify transactions as they arrive, without access to future transactions or global statistics that depend on them. We adopt a strictly *inductive* protocol that forbids any feature leaking information from timesteps $\geq T$ when predicting at timestep T , and we train using walk-forward splits to emulate realistic deployment conditions.

3. Task and the Elliptic++Causal Dataset

This section introduces the strict-inductive task we propose, the dataset modifications it requires, and the engineered features we attach to the regenerated graph. Section 3.1 reviews the original Elliptic dataset; Section 3.2 formalizes the strict-causal contract and the two evaluation protocols we adopt; Section 3.3 describes the Elliptic++Causal regeneration that is the central artifact of this paper; Section 3.4 documents the three families of cross-timestep features we attach to the regenerated graph.

3.1. The Elliptic and Elliptic++ Datasets

We conduct all experiments on the Elliptic dataset (Weber et al., 2019a) and its actor-side extension Elliptic++ (Elmougy & Liu, 2023), summarized in Table 1. Each node represents a single Bitcoin transaction, and directed edges encode the flow of Bitcoin between transactions: an edge from transaction A to transaction B indicates that an output of A was consumed as an input to B . Because a single Bitcoin transaction can involve multiple input and output addresses, the node-level features aggregate across all participating addresses using summary statistics such as means and counts.

Nodes are annotated with 166 features organized into two groups. The first 94 *local* features describe the transaction itself, including its value, fee, number of input and output addresses, and measures of address activity such as average historical volume per address. The remaining 72 *aggregated neighborhood* features summarize the local features of one-hop neighbor transactions via their minimum, maximum, standard deviation, and correlation coefficients. Most feature names are anonymized by Elliptic to protect proprietary detection methods. All transactions are assigned to one of 49 timesteps, each spanning approximately two weeks of blockchain activity.

Of the 203,769 nodes, only 46,564 carry labels: 4,545 are marked *illicit* (associated with scams, malware, darknet marketplaces, and Ponzi schemes) and 42,019 are marked *licit* (exchanges, wallet providers, miners, and other legitimate services), yielding an illicit-to-licit ratio of roughly 1:9 among labeled nodes. The remaining 77% of nodes are unlabeled. This combination of severe class imbalance and partial observability makes the classification task particularly challenging.

3.2. Strict Inductive Detection with Walk Forward Evaluation

We formalize the evaluation regime that all models in this paper are required to satisfy.

Table 1. Summary statistics of the Elliptic dataset.

Property	Value
Nodes (transactions)	203,769
Edges	234,355
Timesteps	49 (~ 2 weeks each)
Illicit labels	4,545 (2.2%)
Licit labels	42,019 (20.6%)
Unlabeled	157,205 (77.2%)
Features per node	166
Local features	94
Aggregated features	72

The strict-causal contract. Let $t(\cdot)$ denote the timestep of a transaction, edge, or feature, and let T be the timestep of a target transaction τ . A model is strictly causal at T if, for every input it consumes when scoring τ , the input’s source timestamp is strictly less than T :

$$\forall \text{ input } i \text{ used to score } \tau \text{ at } T : t(i) < T. \quad (1)$$

The lone exception is the target’s own atomic properties (UTXO structure and on-chain metadata observed at exactly $t(\tau) = T$), which are admissible. Historical labels of past transactions are admissible whenever $t < T$, since by deployment time a fraud team would have classified them. The contract immediately rules out two artifacts of the existing Elliptic++ release in their original form (Section 3.3), and it rules out 72 of the 166 features in Elliptic that aggregate over neighbors whose timestamps may be greater than or equal to T . We refer to the resulting set of strictly causal local features as the *intrinsic 108* feature set.

Evaluation protocol 1: static-causal split. We adopt a strictly temporal split: timesteps 1–29 for training, 30–34 for validation and hyperparameter selection, and 35–49 for testing as a single contiguous block. No information from validation or test timesteps is available during training. Weber et al. (2019a) used a 34/15 train/test split with no held-out validation set. Our three-way split provides unbiased test estimates at the cost of a smaller training window.

This temporal design exposes a well-documented distributional shift around timestep 43, likely corresponding to the sudden closure of a major darknet marketplace (Weber et al., 2019a). Under a random split, test examples from before and after this event are mixed together, masking the model’s inability to handle the regime change. Our split places this shift squarely within the test window, directly measuring robustness to concept drift.

Evaluation protocol 2: walk-forward retraining. The static-causal split is conceptually simple but artificially pessimistic: a deployed system would normally have access

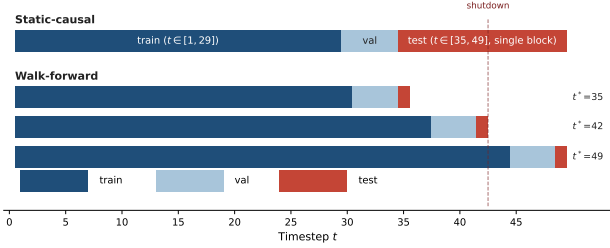


Figure 1. Train/val/test splits under the two evaluation protocols. Top: a single static-causal split with train $t \in [1, 29]$, val $[30, 34]$, and test $[35, 49]$ evaluated as one block. Bottom: walk-forward retraining produces a fresh model for each test timestep T , using train and val windows immediately preceding T . The vertical line at $t = 42.5$ marks the post-shutdown regime change.

to labels from the entire interval $[1, T - 1]$ when scoring T , not only labels from $[1, 29]$. The walk-forward protocol corrects this. For each test timestep $T \in [35, 49]$ we train a fresh model on labelled data from $[1, T - 5]$, with $[T - 4, T - 1]$ held out for validation, and predict transactions at exactly $t = T$. Predictions are pooled across all T and evaluated jointly. The protocol simulates a deployed anti-money laundering system that retrains nightly as freshly labelled transactions arrive, and is what we view as the baseline for any reported number on this benchmark.

3.3. The Elliptic++Causal Dataset

The Elliptic++ release (Elmougy & Liu, 2023) extends Elliptic by introducing wallet addresses as a second node type, yielding a heterogeneous graph with three relation types: tx $\leftrightarrow$ tx edges (the 234K within-timestep flows already present in Elliptic), wallet $\leftrightarrow$ tx bipartite edges (1.27M, recording which wallets appeared as inputs or outputs of which transactions), and wallet $\leftrightarrow$ wallet edges (2.87M, recording wallet co-occurrence). Two of these actor-side artifacts are of immediate interest for inductive learning but unusable in their original form: a wallet feature file (wallets_features.txt) with one row per (wallet, timestep) pair, and the wallet-wallet edge file (AddrAddr_edgelist.txt). Both leak future information at every timestep $T < 49$. We describe the leakage and our regeneration of each file in turn.

Wallet feature leakage. The original wallet feature data contains 56 numerical features per (wallet, timestep) row, intended to summarize wallet-level activity through that timestep. Inspection of the file reveals, however, that all rows for the same wallet are byte-identical: every row for wallet w contains lifetime aggregates, counts, volumes, and ratios computed over w ’s entire activity through timestep 49, regardless of which T the row is indexed at. A model that consumes such a row when scoring a transaction at $t = T$

implicitly observes activity that has not yet occurred. Even simple summary statistics like “number of transactions sent by w ” encode information from $\tau > T$ in their original form.

Our regeneration, wallets_features_causal.csv, rewrites this file so that the row for (w, T) contains aggregates computed only from events with $\tau \leq T$. Concretely, we sweep the (wallet, transaction) co-incidence list in chronological order and, at each T , record cumulative counts, volumes, fan-in/fan-out ratios, gap statistics, and decayed-illicit scores. The resulting file has the same schema as the original but is admissible at every T : any model that consumes the row for (w, T) sees only w ’s activity through T .

Wallet-wallet edge leakage. The original AddrAddr_edgelist.txt contains 2,868,964 directed wallet-wallet edges with no timestamp annotations. An edge (w_1, w_2) encodes that w_1 and w_2 co-transacted at some point during the entire lifetime of the dataset, with the directed orientation indicating that w_1 appeared on the input side and w_2 on the output side of a shared transaction. There is no way to tell, from the original file alone, when this co-transaction first occurred. Loading the file into a heterogeneous graph at any timestep $T < 49$ therefore admits edges that record co-transactions yet to occur.

Our regeneration, AddrAddr_edgelist_causal.csv, attaches a first_t field to each edge equal to the earliest timestep at which the two wallets co-transacted in the appropriate input/output orientation. We compute this by joining the (wallet, transaction) co-incidence list against the transaction timestep table: for each transaction τ at time $t(\tau)$, we enumerate every (input-wallet, output-wallet) pair on τ and record an edge with $\text{first_t} = \min(\text{first_t}, t(\tau))$. The resulting file is identical in row count and schema to the original except for the additional first_t column. In a strictly inductive temporal model, edge e is admissible at time T only if $\text{first_t}(e) \leq T$. Heterogeneous graph snapshots can therefore be built incrementally: $\text{snapshot}(T)$ contains exactly those wallet-wallet edges with $\text{first_t}(e) \leq T$, plus the bipartite edges and tx-tx edges incident to transactions with timestep at most T .

Released artifacts. We release three CSV files together as the Elliptic++Causal dataset: the regenerated wallet feature file (wallets_features_causal.csv, 56 cumulative features per row), the regenerated wallet-wallet edge file (AddrAddr_edgelist_causal.csv, 2.87M edges with first_t annotations), and a precomputed (wallet, transaction) co-incidence table that materialises the bipartite graph in causality-aware form. Combined with the unchanged transaction-level Elliptic files, this is sufficient to construct a fully causal heterogeneous Bitcoin transaction

Table 2. Elliptic++Causal: nodes, edges, and feature dimensions of the regenerated heterogeneous graph.

Property	Value
Transaction nodes	203,769
Wallet nodes	822,942
Tx-tx edges (within timestep)	234,355
Wallet $\leftrightarrow$ tx edges (bipartite)	1,268,051
Wallet $\leftrightarrow$ wallet edges (causal)	2,868,964
Tx features (raw + Elliptic++)	182
Tx features (intrinsic 108 only)	108
Wallet features (causal)	56

graph at every timestep $T \in [1, 49]$. Table 2 summarizes the resulting structure.

3.4. Engineered Cross-Timestep Features

The Elliptic++Causal regeneration removes the leakage in the original Elliptic++ wallet-side data but does not, on its own, supply additional cross-timestep signal. To inject such signal, we engineer three families of strictly causal cross-timestep features per transaction, all of which operate within the strict-causal contract of equation (1) by construction.

Wallet trajectory features (103 features per transaction). For each wallet incident to a transaction at time T , we summarize the wallet’s complete prior history with $\tau < T$. The construction yields 103 features per transaction, computed in two passes: a chronological sweep of every (wallet, transaction) co-incidence builds per-wallet history arrays sorted by time, and a per-target lookup uses `numpy.searchsorted` with side "left" to recover the wallet’s prior state with strict $\tau < T$ semantics. The features fall into five sub-families: counts (number of prior transactions, distinct timesteps active, distinct counterparties); cumulative volumes (BTC sent, received, fees paid); decayed-illicit scores (sum of $\exp(-\lambda(T - \tau)) \mathbb{I}[\text{tx at } \tau \text{ illicit}]$ over the wallet’s prior history, for several λ values); recency aggregates (time since first and most-recent activity, gaps); and fan-structure features (in-degree, out-degree, asymmetry ratios). Per-side aggregates over input wallets and output wallets are concatenated to form the per-transaction feature vector.

Cross-timestep money-flow pair features (17 features per transaction). We construct an auxiliary directed graph in which transactions τ_1 and τ_2 with $t(\tau_1) < t(\tau_2)$ are connected if they share at least one wallet, with the edge labelled by which side of each transaction the wallet appears on (input-input, input-output, output-input, output-output). Every edge carries the timestep difference $\Delta t = t(\tau_2) -$

$t(\tau_1) > 0$, so causal slicing is just a Δt filter at scoring time. We extract 17 per-target features: counts of incoming edges by role; aggregates over the Δt distribution; decayed-illicit mass weighted by source-transaction labels with $\tau < T$; and aggregates over source-transaction features.

Illicit seeded personalized PageRank (5 features per transaction). The most novel of our features is a personalized PageRank score computed at every prediction step on the cumulative bipartite graph, with the teleport vector concentrated on the historically illicit subgraph. The construction at time T proceeds in three steps. We first form the bipartite graph G_T over wallets and transactions, restricted to edges with host-tx $\tau < T$. We then set the teleport vector r_T to a uniform mass over the set of transactions labelled illicit at $\tau < T$ and zero elsewhere. Finally we iterate the PPR update

$$\pi^{(k+1)} = (1 - \alpha) P^\top \pi^{(k)} + \alpha r_T \quad (2)$$

to convergence, with $\alpha = 0.15$ and P the row-stochastic transition matrix on G_T . Each transaction τ at $t = T$ then receives five PPR-derived features: its own equilibrium PPR mass, plus the min, max, mean, and sum of PPR mass over its incident wallets. These features capture multi-hop reachability from the historical illicit subgraph that pure one-hop trajectory features cannot express.

Feature dimensionality summary. Table 3 summarizes the resulting feature representations across our model hierarchy. Tree-based models on engineered features consume the 233-dimensional union of the intrinsic 108 features and the three engineered families (we refer to this set as $C1$ throughout). Vanilla GCN/SkipGCN baselines consume the original 166 features for direct comparison with prior work. Heterogeneous GNNs consume up to 285 dimensions per transaction node (the 182-dimensional raw Elliptic++ features plus the 103 trajectory features), with wallet nodes carrying the 56-dimensional causal wallet features. The relational graph transformer additionally concatenates the 5 PPR features for a 290-dimensional transaction representation.

4. Methods

We benchmark four model families on the proposed task: tree-based classifiers on engineered features (Section 4.1), vanilla and skip-connected GCN baselines on the original 166 features (Section 4.2), and three heterogeneous GNN architectures on the Elliptic++Causal graph (Section 4.3). All models are trained and evaluated under both the static-causal split and the walk-forward protocol of Section 3.2.

Table 3. Feature representations across model stages. *CI* is the headline 233-dimensional feature set used by tree models.

Stage	Dim.	Features included
Intrinsic 108	108	strict-causal local features
Original Elliptic	166	94 local + 72 aggregated
+ Elliptic++ attributes	182	+ 16 named tx features
+ Wallet trajectories	285	+ 103 causal wallet history
+ PageRank	290	+ 5 illicit-proximity scores
<i>CI</i> (tree headline)	233	108 + traj + tx + PR

4.1. Tree Baselines on Engineered Features

Random Forest and gradient-boosted-tree models on tabular features have historically dominated the Elliptic benchmark (Weber et al., 2019a). We benchmark two such models under the strict-causal contract: a Random Forest classifier with `n_estimators=500`, `max_features="sqrt"`, and `class_weight="balanced"`, and a LightGBM classifier with `num_leaves=63`, `learning_rate=0.05`, `n_estimators=500`, and balanced class weights. Both consume the 233-dimensional *CI* feature set (intrinsic 108 plus the engineered trajectory, pair, and PPR features from Section 3.4). Hyperparameters are selected on the validation window $t \in [30, 34]$. Under walk-forward retraining we fit a fresh model from scratch at each test timestep T on data from $[1, T-1]$, using $[T-4, T-1]$ as a temporal validation window for early stopping (LightGBM only).

Threshold selection. A binary classifier produces a continuous illicit probability $\hat{p}(\tau) \in [0, 1]$ for each transaction, which is converted to a hard prediction by comparing against a threshold θ . The choice of θ does not change what the model learned during training; it only changes how the continuous output is discretized into class labels. Throughout the paper we report two columns: F1 at a calibrated threshold θ^* obtained by sweeping θ on the validation window and selecting whichever value maximises validation F1, and F1@0.5 at the default threshold without validation tuning. Reporting both isolates how much of any reported F1 comes from the model versus from threshold tuning, which matters because the validation window’s illicit rate (16.8%) differs substantially from the test horizon’s (6.5%), thus making validation tuned thresholds an unreliable proxy for test-time optimal thresholds, a point we further discuss in Section 5.

The strict no-graph floor we report against throughout the paper is the Random Forest on the intrinsic 108 features, which scores illicit-class F1 of 0.802 under the static-causal split.

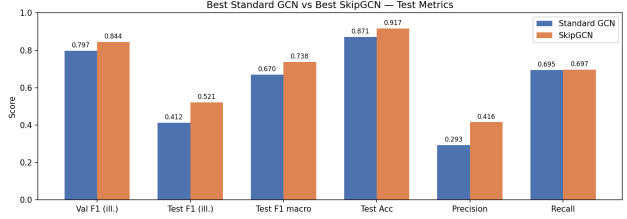


Figure 2. Test set comparison between the best standard GCN and SkipGCN models. SkipGCN improves across all metrics, with the largest gains in precision (+0.123) and illicit-class F1 (+0.109).

4.2. GNNs vs Baseline

Before introducing our temporal feature engineering or walk-forward training protocol, we first explore a simpler question: can graph neural networks, trained in the standard transductive setting with the original 166 Elliptic features, outperform the Random Forest baseline? Prior work has consistently reported that GNNs underperform on this benchmark (Weber et al., 2019a), but the reasons remain underexplored. We investigate this by training both a standard GCN (Kipf & Welling, 2017) and a SkipGCN variant, which augments each GCN layer with a residual connection from the input features using the temporal train/test split described in Section 3.2 but without any of our engineered features or walk-forward retraining. This isolates the contribution of graph structure alone.

We sweep over the number of layers $L \in \{2, 3, 4\}$, hidden dimension $H \in \{64, 128, 256\}$, and dropout rate $d_o \in \{0.3, 0.5\}$, training each configuration for 600 epochs. Figure 7 shows the training loss and validation F1 (illicit class) curves across all configurations for the standard GCN. Training loss converges smoothly regardless of architecture, but validation F1 varies substantially across hyperparameter settings, with the best configurations plateauing near 0.80, already below the Random Forest baseline of 0.802.

Figure 2 compares the best standard GCN and SkipGCN on the test set (timesteps 34–49). SkipGCN improves over the standard GCN on every metric: illicit-class F1 rises from 0.412 to 0.521, precision from 0.293 to 0.416, and accuracy from 0.871 to 0.917. Recall remains nearly identical (0.695 vs. 0.697), indicating that skip connections reduce false positives rather than improving detection coverage. The gains confirm that preserving the original feature signal via residual connections is important when the input features are already highly informative: deep message-passing without skip connections degrades the useful signal through oversmoothing.

However, even the best SkipGCN (F1 = 0.521) falls far short of the Random Forest baseline (F1 = 0.802). Figure 6 reveals the primary failure mode: both GCN variants suffer

a near-complete collapse after timestep 43, with illicit-class F1 dropping to near zero. In the pre-shift period (timesteps 35–42), SkipGCN performs reasonably well, but neither architecture recovers after the distributional shift. The Random Forest, by contrast, degrades more gracefully across the regime boundary because it operates on each transaction independently rather than propagating information through a graph whose structure changes across timesteps.

These results motivate the remainder of our approach. Since vanilla GNNs cannot match the Random Forest baseline, we shift focus to temporal graph features (Section 3.4) and walk-forward training (Section 3.2).

4.3. Heterogeneous GNNs on the Causal Graph

The Elliptic++Causal regeneration enables, for the first time, training graph neural networks under the strict-causal contract on a heterogeneous Bitcoin transaction graph that includes wallet nodes and wallet-wallet edges. We evaluate three architectures from this family: a heterogeneous GCN with per-relation GraphConv aggregators, a heterogeneous GAT with per-relation GATConv, and a heterogeneous GraphSAGE with per-relation max-pool SAGEConv and L2 normalization. All three share the same per-target sampled-forward training contract: for each labelled training transaction at time k , we build a snapshot graph containing only edges with timestamp $\leq k$ (using the `first_t` field from `AddrAddr_edgelist_causal.csv` for wallet-wallet edges), sample a 2-hop neighborhood with `NeighborLoader`, and compute loss only on the seed transaction. This contract is strictly stronger than the conventional “inductive GNN” protocol: the model never sees test-time edges, but more importantly it never sees future- k edges within the training window either.

We isolate the contribution of each part of the regenerated graph through four edge-and-feature ablations. The *bipartite* ablation uses only wallet-tx edges and the raw 182-dimensional tx features. The *+wallet-wallet* ablation adds the causal AddrAddr edges from Section 3.3 (gated by their `first_t` field). The *full* ablation additionally includes the within-timestep tx-tx edges. The *full+traj* ablation uses the same full edge set but augments the tx features with the 103 trajectory features from Section 3.4, for a 285-dimensional tx representation. Wallet nodes carry the 56-dimensional causal wallet features in all four ablations.

Hyperparameters are fixed across the grid: hidden dimension 128, 3 layers, four attention heads (GAT), dropout 0.4, AdamW with learning rate 5×10^{-4} and weight decay 5×10^{-4} , gradient clipping at 1.0, and bf16 mixed-precision training on a single A100 GPU. Note that due to memory constraints, we also used Torch’s Neighbor Loader which implements snowball sampling to reduce GPU RAM usage. Threshold calibration uses the validation window

$t \in [30, 34]$ in the strict-inductive setting and the rolling window $[T - 4, T - 1]$ in the walk-forward setting. We report illicit-class F1 at both the calibrated and the 0.5 threshold, AUC, and PR-AUC. For more detail on our parameter choices for both walk-forward and strict-inductive models, see 6, 7, and 8.

5. Results Discussion

This section reports the first systematic baselines on the strict-inductive task and Elliptic++Causal dataset proposed in Sections 3.2–3.3. Section 5.1 reports strict-inductive results across all four model families. Section 5.2 adds walk-forward retraining and reports the headline pooled metrics. Section 5.3 decomposes the combined lift into independent main effects and a synergy term that only becomes visible under the proposed task framing. Section 5.4 examines per-timestep behaviour around the post-shutdown cliff at $t = 43$.

5.1. Strict-Inductive Results

Table 4 summarises strict-inductive performance across the model families. The strict no-graph floor is the Random Forest on the intrinsic 108 features, which scores 0.802 illicit-class F1. The original 166-feature RF, by contrast, scores 0.806 on the same temporal split, a difference of only 0.004, indicating that the 72 within-timestep aggregated features contribute essentially no causal signal once the strict-causal contract is enforced. Adding our 103 trajectory features to the strict baseline lifts RF to 0.810, the 17 pair features to 0.812, and the 5 PPR features to 0.815 (the C1 configuration).

The heterogeneous GNN block in Table 4 is, to our knowledge, the first reported set of strict-inductive numbers for graph neural networks on a wallet-aware Bitcoin transaction graph. The pattern across ablations is striking and consistent: adding wallet-wallet edges (*+wallet-wallet*) lifts Hetero-GCN by +0.075 F1, but the further addition of within-timestep tx-tx edges (*full*) provides essentially no marginal gain, and in some cases hurts. The single largest lever is the trajectory features: moving from *full* to *full+traj* adds +0.158 to +0.178 F1 across all three architectures, with Hetero-SAGE reaching F1 = 0.668 and AUC = 0.935. This is the central empirical finding of the GNN exploration: the engineered trajectory features generalize across model families, lifting both tree models and graph neural networks substantially. The dataset contribution is what closes most of the gap between graph models and feature-engineered baselines, not the choice of GNN architecture.

The vanilla GCN/SkipGCN baselines included for reference reach F1 = 0.412 and 0.521 under transductive evaluation on the original 166 features (Section 4.2). Even the strongest

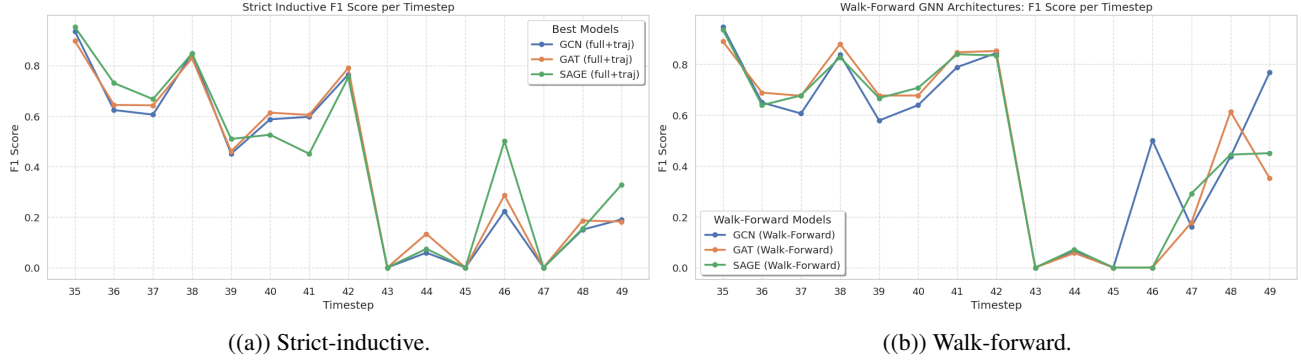


Figure 3. Per-timestep F1 on the illicit class for GCN, GAT, and GraphSAGE. **Left:** strict-inductive protocol, all three architectures collapse uniformly after the regime change at timestep 43. **Right:** walk-forward protocol, refitting on a rolling window partially recovers post-shift performance. The collapse is driven by distribution shift, not architecture, and is largely remediable by retraining.

of these falls well short of the heterogeneous Hetero-SAGE on the regenerated causal graph, suggesting that wallet-side structure, absent from the original Elliptic and unusable in the original Elliptic++, carries non-negligible signal once it is made causally admissible.

5.2. Walk Forward Results

Table 5 reports walk-forward results on the strongest configurations from each model family. The headline number of this paper is the walk-forward LightGBM on C1, which reaches pooled illicit-class F1 = 0.856, AUC = 0.975, and PR-AUC = 0.899 on the test horizon $t \in [35, 49]$. Walk-forward retraining lifts the heterogeneous GNNs as well: the best Hetero-GCN (*full+traj*) reaches pooled F1 = 0.723 (up from 0.656 in the strict-inductive setting), Hetero-GAT reaches F1 = 0.714, and Hetero-SAGE reaches F1 = 0.690. Across the GNN family, walk-forward adds roughly +0.03 to +0.07 F1 on top of the strict-inductive baseline. Tree models receive a larger lift of +0.041 F1 from walk-forward (Random Forest C1 from 0.815 to 0.824; LightGBM C1 from 0.800 to 0.856).

5.3. Why the Task Framing Matters: The Synergy Term

The two-by-two design embedded across Tables 4 and 5 (graph features yes/no, walk-forward yes/no) lets us decompose the lift cleanly. Holding evaluation static and adding C1 graph features gives +0.013 F1 (0.802 to 0.815). Holding the feature set at the no-graph 108 and switching to walk-forward LightGBM gives +0.003 F1 (0.802 to 0.806). Combining both gives +0.054 F1 (0.802 to 0.856), so the synergy term is

$$\Delta_{\text{synergy}} = 0.054 - 0.013 - 0.003 = +0.038 \quad (3)$$

illicit-class F1, larger than either main effect by an order of magnitude. Figure 4 renders the decomposition as a waterfall.

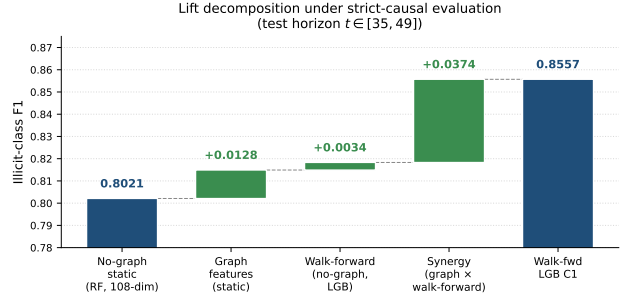


Figure 4. Decomposition of the lift from the strict no-graph floor (left, F1 = 0.802) to the headline walk-forward LightGBM C1 model (right, F1 = 0.856). Graph features alone contribute +0.013, walk-forward retraining alone contributes +0.003, and their interaction contributes +0.037. The synergy term is the largest single component and is invisible to any evaluation that uses static splits alone.

The interpretation is methodological. Under the static split, the model fixes its mapping from graph features to predictions at training time on $t \leq 29$ and cannot adapt; the marginal benefit of the engineered graph features is therefore small. Under walk-forward retraining, the model is re-fit at every T on labels through $T - 1$, and is free to re-weight or even invert features whose distribution has shifted. Graph features, which are the most distribution-sensitive part of the feature set, are the primary beneficiary of this adaptive capacity. The synergy term represents this, showing how our walk-forward and inductive features beat models who use static splits. This is the most concrete sense in which the proposed task framing matters: under the standard transductive or static-causal protocol, our engineered graph features would appear to add only 0.013 F1 over the baseline, suggesting they are at best a marginal improvement; under the deployment-faithful walk-forward protocol, they unlock an additional 0.037 F1 on top, suggesting they are the central engineering contribution.

Table 4. Strict-inductive results on the test horizon $t \in [35, 49]$, illicit-class metrics. F1 is reported at the validation-calibrated threshold and at threshold 0.5; AUC and PR-AUC are threshold-free.

Model	Features	F1	F1@0.5	AUC	PR-AUC
<i>Tree baselines (this work):</i>					
Random Forest	108 (intrinsic)	0.802	0.802	0.903	0.786
Random Forest	233 (C1)	0.815	0.815	0.921	0.807
LightGBM	233 (C1)	0.800	0.800	0.928	0.804
<i>Vanilla GCN baselines (transductive 166-d):</i>					
GCN (Kipf & Welling, 2017)	166	0.412	—	—	—
SkipGCN	166	0.521	—	—	—
<i>Heterogeneous GNNs on Elliptic++Causal:</i>					
Hetero-GCN	bipartite	0.448	0.471	0.876	0.286
Hetero-GAT	bipartite	0.580	0.381	0.892	0.427
Hetero-SAGE	bipartite	0.467	0.419	0.879	0.325
Hetero-GCN	+wallet-wallet	0.523	0.440	0.892	0.402
Hetero-GAT	+wallet-wallet	0.522	0.464	0.891	0.327
Hetero-SAGE	+wallet-wallet	0.478	0.473	0.875	0.370
Hetero-GCN	full	0.478	0.468	0.872	0.299
Hetero-GAT	full	0.507	0.475	0.887	0.339
Hetero-SAGE	full	0.510	0.492	0.889	0.494
Hetero-GCN	full+traj	0.656	0.646	0.919	0.682
Hetero-GAT	full+traj	0.667	0.610	0.930	0.690
Hetero-SAGE	full+traj	0.668	0.644	0.935	0.711

Table 5. Walk-forward results, pooled across $t \in [35, 49]$. All models use the strongest feature configuration from Table 4.

Model	Features	F1	AUC	PR-AUC
Random Forest	233 (C1)	0.824	0.972	0.876
LightGBM (no graph)	108	0.806	0.970	0.871
LightGBM (C1)	233 (C1)	0.856	0.975	0.899
Hetero-GCN	full+traj	0.723	0.954	0.752
Hetero-GAT	full+traj	0.714	0.947	0.751
Hetero-SAGE	full+traj	0.690	0.928	0.729

5.4. Per-Timestep Behaviour and the Cliff

The pre-shutdown horizon $t \in [35, 42]$ is essentially solved by every walk-forward model: walk-forward LightGBM C1 reaches F1 above 0.79 on every pre-shutdown timestep, with peaks of 0.973 at $t = 35$ and 0.971 at $t = 36$. The post-shutdown horizon $t \geq 43$ is where the ablations separate sharply (Figure 5). Static RF C1 collapses to F1 of 0.146 averaged across the cliff region; walk-forward LightGBM C1 holds at F1 = 0.536. At $t = 49$, where the model has been retrained on labels through $t = 48$ (post-shutdown distribution), walk-forward LightGBM C1 reaches F1 = 0.964. The cliff is not eliminated (timesteps $t = 45$ and $t = 47$ still classify at F1 = 0 because they contain only 5 and 22 illicit examples respectively) but the working-regime gap that opens at $t \geq 43$ is substantially narrowed.

The walk-forward GNNs follow a similar pattern: pre-shutdown F1 in the range 0.6–0.94 across architectures,

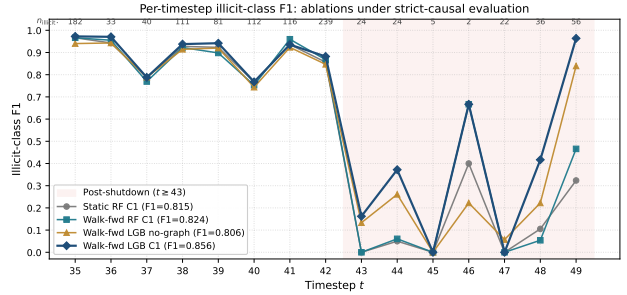


Figure 5. Per-timestep illicit-class F1 across four ablations of feature set and evaluation regime. The pre-shutdown horizon $t \in [35, 42]$ is essentially solved by every model. The post-shutdown horizon (shaded, $t \geq 43$) is where graph features and walk-forward retraining pull apart. Walk-forward LightGBM with the C1 feature set (dark blue) reaches F1 = 0.964 at $t = 49$, against a static-RF F1 of 0.324.

near-zero F1 at $t = 43$ ($n_{\text{illicit}} = 24$), and partial recovery on later post-shutdown timesteps as the retraining window incorporates post-shutdown labels. Hetero-GCN *full+traj* reaches per-timestep F1 of 0.768 at $t = 49$.

6. Conclusion

Our work proposes three methodological changes to the standard Elliptic benchmark: a strict-inductive task with walk-forward evaluation that mirrors deployment, the Elliptic++Causal regeneration that makes the heterogeneous

Bitcoin transaction graph admissible under that task, and three families of strictly causal cross-timestep features that together close most of the gap between graph models and feature-engineered baselines. The headline tree model on this task reaches illicit-class F1 of 0.856 (AUC 0.975), and the strongest heterogeneous GNN reaches F1 of 0.723 (AUC 0.954). Across both model families, the central engineering contribution is the trajectory feature family computed on the regenerated wallet-wallet graph: it lifts heterogeneous GNN F1 by roughly +0.16 and contributes +0.005 to tree-model F1, with the larger tree-model gains coming from synergy with walk-forward retraining.

Several limitations of the current work suggest natural extensions. First, although walk-forward retraining narrows the post-shutdown cliff substantially, two cliff-region timesteps ($t = 45, 47$) still classify at $F1 = 0$ because they contain only single-digit numbers of illicit examples; ensemble methods that pool predictions across recent timesteps may help. Second, our negative results (vanilla GCN/SkipGCN underperformance, modest contribution of within-timestep tx-tx edges, unreliable threshold calibration on the temporal validation window) suggest that gradient-boosted trees on engineered features remain the strongest single-model approach to this task, but we have not exhausted the space of imbalance-aware GNN methods. Three directions in particular are worth pursuing: GraphENS-style ego-network synthesis (Park et al., 2022), the self-supervised pretraining of Inspection-L, and one-class objectives. Combined with the walk-forward training schedule we propose, any of these could plausibly close the gap to the LightGBM headline. We release the Elliptic++Causal dataset and the code to reproduce every number in this paper, and we hope it provides a more honest substrate for that line of work.

References

- Elmougy, Y. and Liu, L. Demystifying fraudulent transactions and illicit nodes in the Bitcoin network for financial forensics. *arXiv preprint arXiv:2306.06108*, 2023.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations (ICLR)*, 2017.
- Park, J., Song, J., and Yang, E. GraphENS: Neighbor-aware ego network synthesis for class-imbalanced node classification. In *International Conference on Learning Representations (ICLR)*, 2022.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Weber, M., Domeniconi, G., Chen, J., Weidele, D. K. I., Bellei, C., Robinson, T., and Leiserson, C. E. Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics. In *KDD Workshop on Anomaly Detection in Finance*, 2019a.
- Weber, M. et al. The Elliptic data set: Opening up machine learning on the blockchain. *arXiv preprint arXiv:1908.02591*, 2019b.

A. Appendix

A.1. Parameter Choices

Table 6. Strict-inductive GNN hyperparameters.

Model	Hidden	Layers	Heads	Dropout	LR	Weight decay
GCN	128	3	—	0.4	5×10^{-4}	5×10^{-4}
GAT	128	3	4	0.4	5×10^{-4}	5×10^{-4}
GraphSAGE	128	3	—	0.4	5×10^{-4}	5×10^{-4}

Adam, batch 2048, fanout [20, 15, 15], 50 epochs, patience 12, grad clip 1.0.

Table 7. Walk-forward heterogeneous GNNs.

Model	Hidden	Layers	Heads	Dropout	LR	Weight decay
GCN	128	3	—	0.4	5×10^{-4}	5×10^{-4}
GAT	128	3	4	0.4	5×10^{-4}	5×10^{-4}
GraphSAGE	128	3	—	0.4	5×10^{-4}	5×10^{-4}
CausalGNN	64	2	—	0.2	1×10^{-3}	1×10^{-5}

Adam, batch 2048, fanout [20, 15, 15], 10 epochs.

Table 8. Walk-forward tree-based models.

Model	n_{est}	Leaves / depth	LR	Min child	Class weight
LightGBM (grid)	2000	{31, 63, 127}	{0.05, 0.10}	{10, 20}	—
LightGBM	2000	63 leaves	0.10	20	—
XGBoost	2000	depth 6	0.05	—	—
RandomForest	200	—	—	—	balanced

LightGBM grid selected by validation PR-AUC.

A.2. Homogeneous GCN Ablation

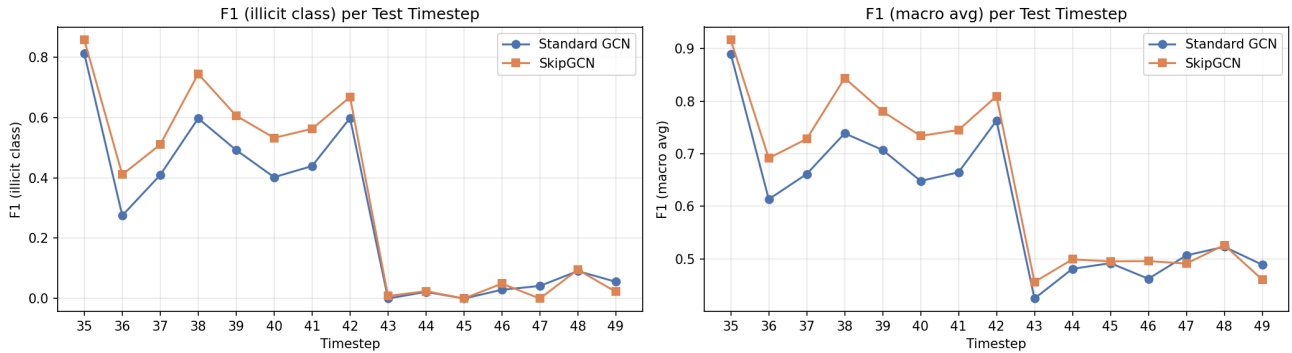


Figure 6. Per-timestep performance on the test set. **Left:** F1 on the illicit class. **Right:** Macro-averaged F1. Both GCN variants collapse after the regime change at timestep 43, with SkipGCN showing modest advantages in the pre-shift period (timesteps 35–42).

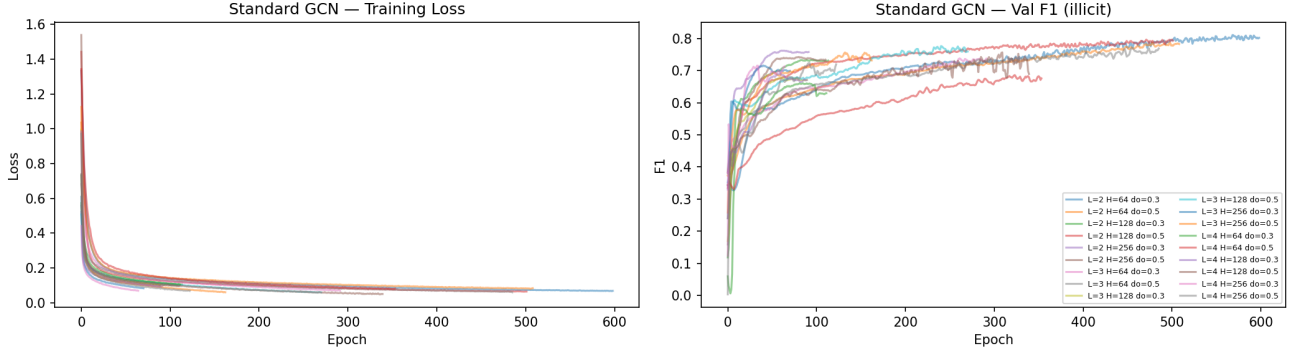


Figure 7. Standard GCN training dynamics across hyperparameter configurations ($L \in \{2, 3, 4\}$, $H \in \{64, 128, 256\}$, $d_o \in \{0.3, 0.5\}$). **Left:** Training loss converges consistently across settings. **Right:** Validation F1 on the illicit class shows greater sensitivity to architecture choices, with the best configurations plateauing near 0.80.

A.3. Strict-Inductive Results

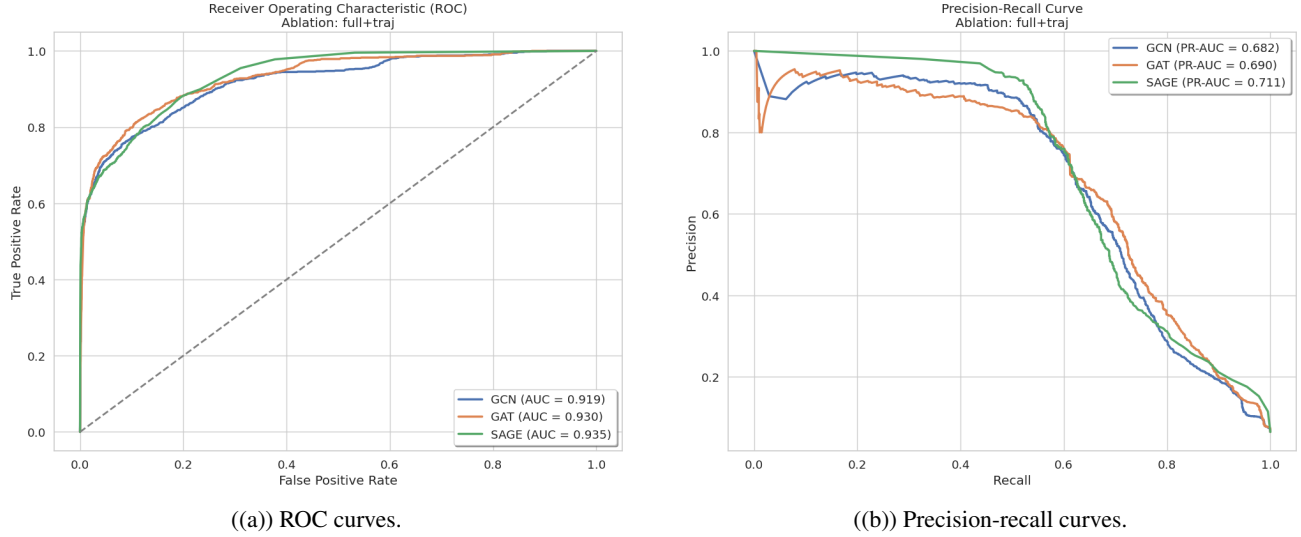


Figure 8. Strict-inductive ROC and precision-recall curves for GCN, GAT, and GraphSAGE on the full+traj feature set.

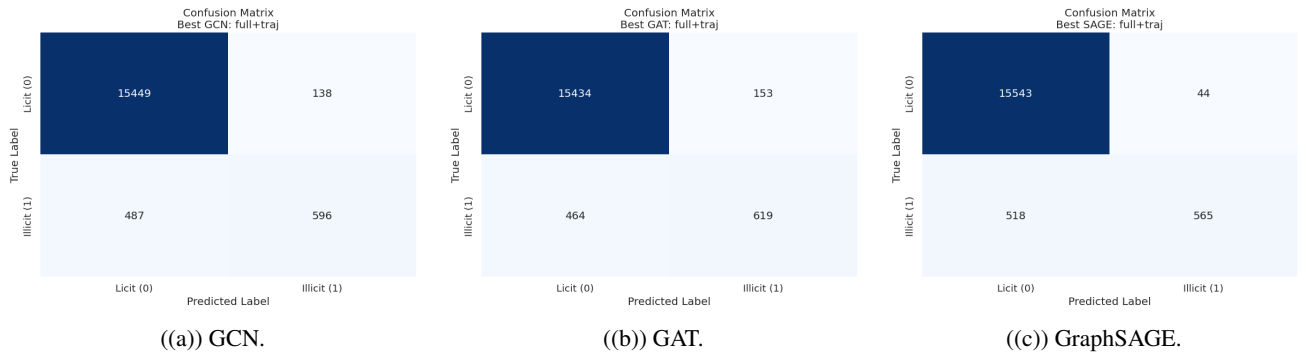


Figure 9. Confusion matrices for the best strict-inductive GCN, GAT, and GraphSAGE models on the full+traj feature set.

A.4. Strict-Inductive vs. Walk-Forward

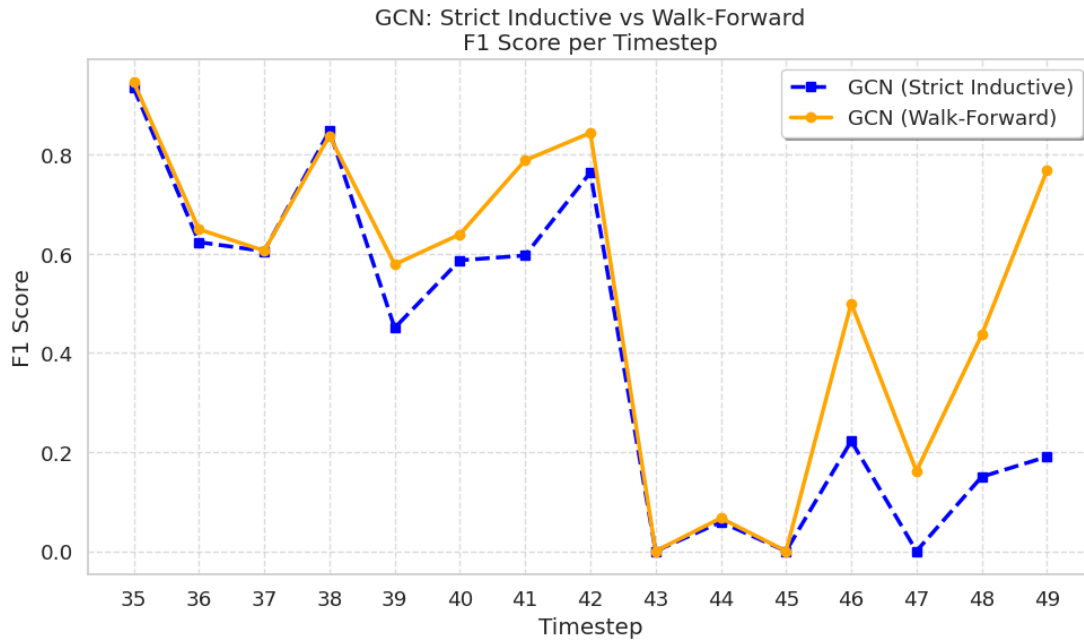


Figure 10. GCN per-timestep F1: strict-inductive vs. walk-forward. Refitting per window recovers most of the post-step-42 collapse.

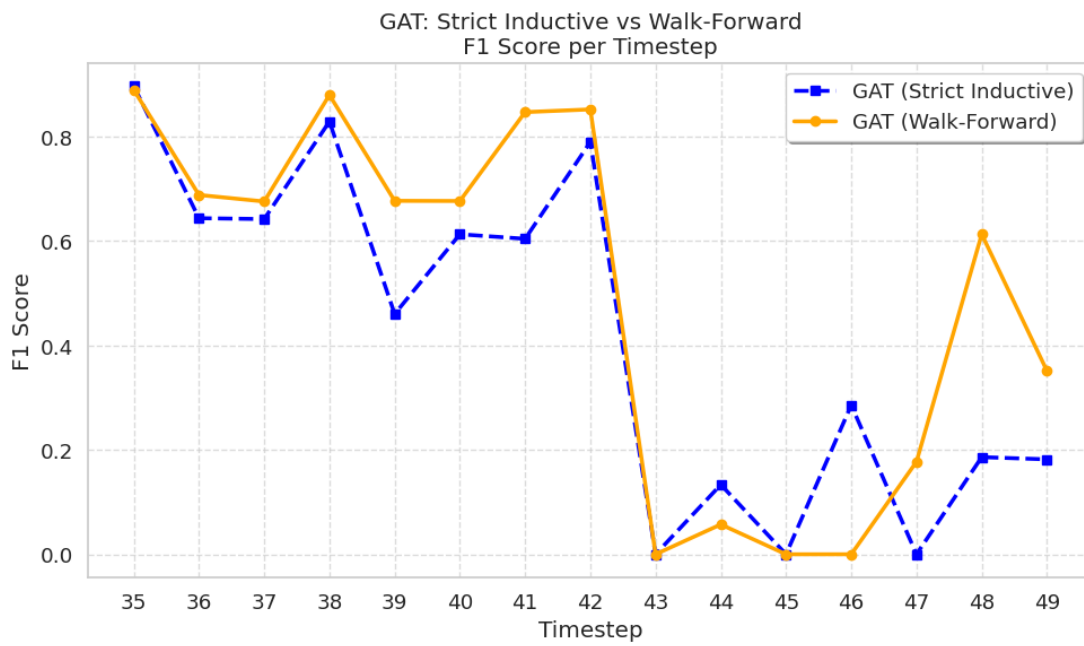


Figure 11. GAT per-timestep F1: strict-inductive vs. walk-forward.

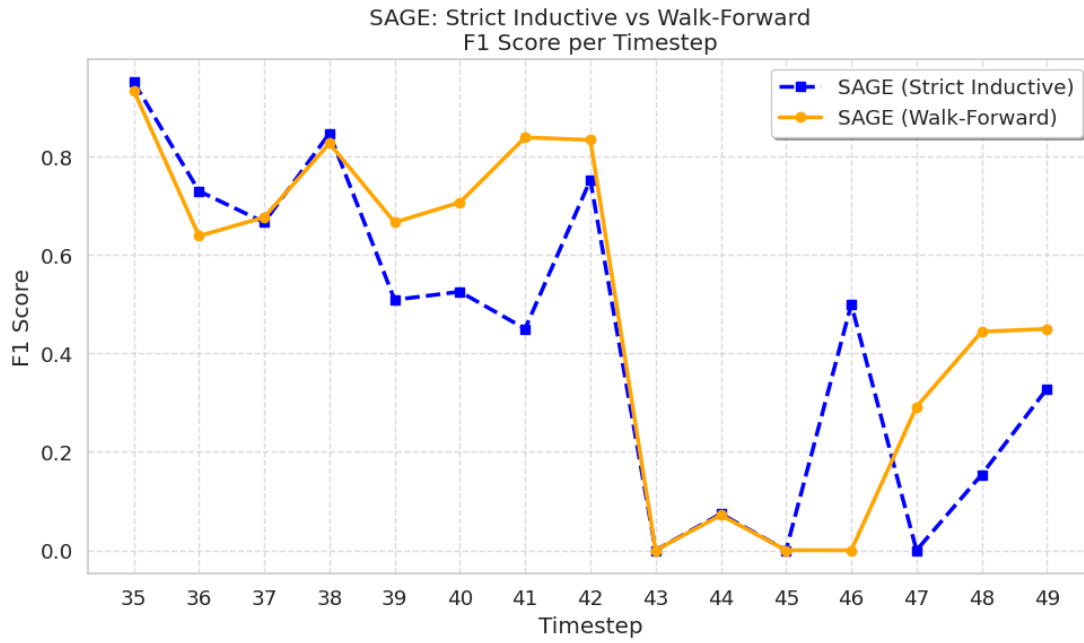


Figure 12. GraphSAGE per-timestep F1: strict-inductive vs. walk-forward.

A.5. Walk-Forward Results

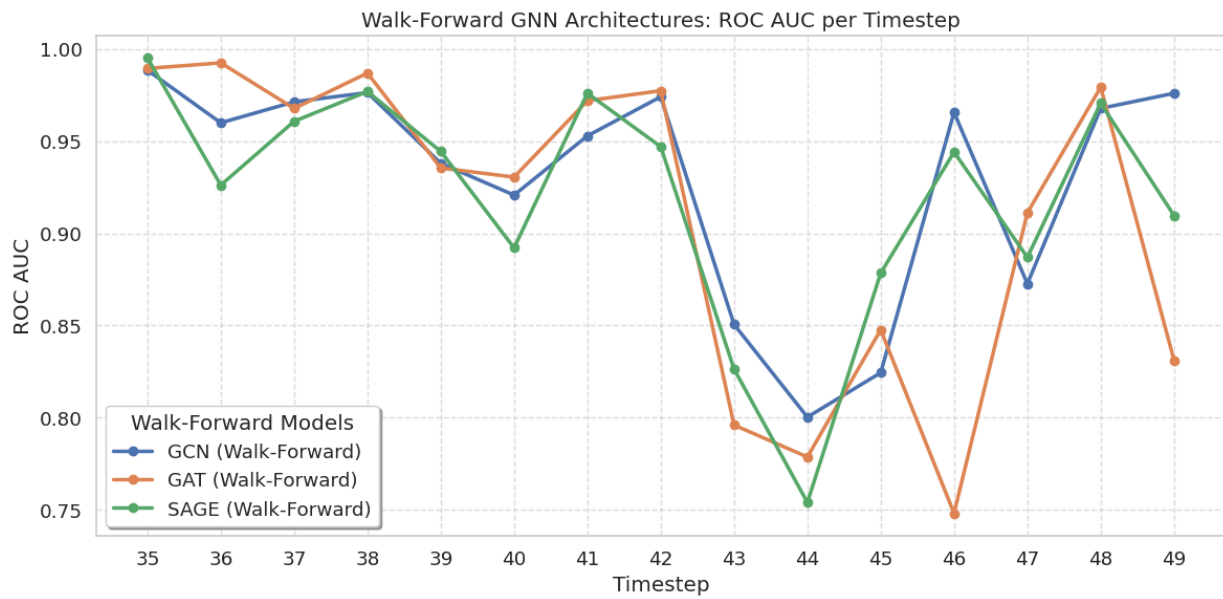


Figure 13. Per-timestep ROC AUC under the walk-forward protocol.

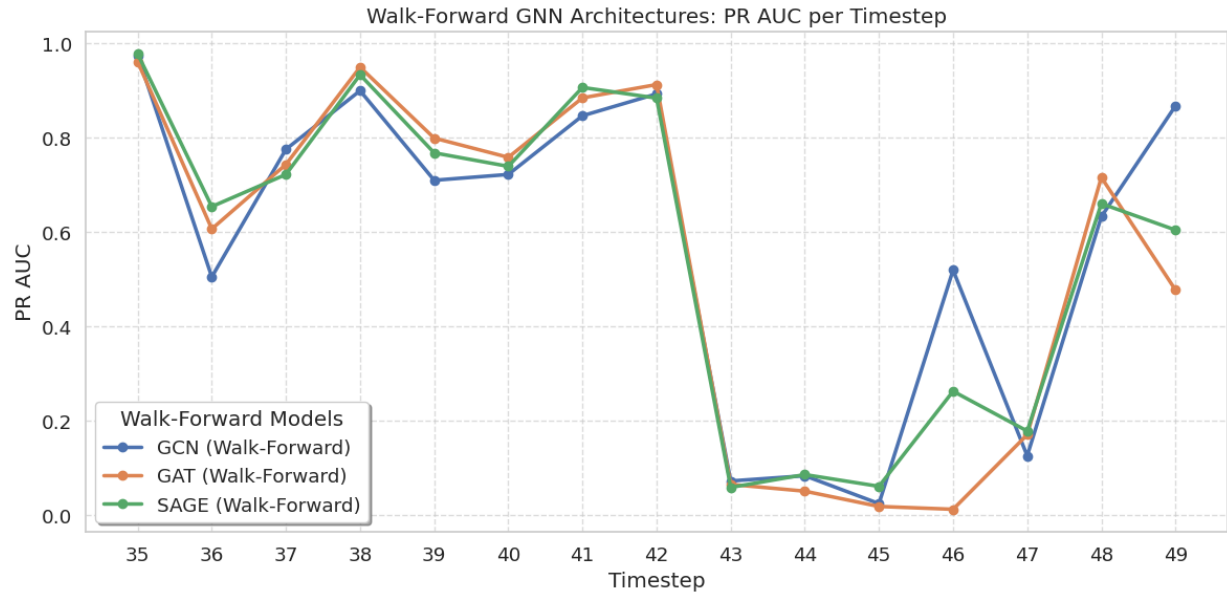


Figure 14. Per-timestep PR AUC under the walk-forward protocol. The post-step-42 collapse is most visible in PR AUC, since the illicit base rate drives the precision floor.